

From a Local AI Agent to a Cloud MCP Service: Stage 2 Evolution of the Python Code Tool Interpreter for Chart Creation

Md Ashiqur Rahman Md Sohel Rana
Master's Program in Information Technology
Frankfurt University of Applied Sciences, Germany
Cloud Computing Project, Project ID ML-25-26-05
Supervisor: Prof. Damir Dobric

Abstract—Stage 1 of this project implemented a local AI-assisted data-visualization agent that translated natural-language requests into Python/Matplotlib code, validated the generated code, executed it through a controlled child process, and returned chart images. Although the local design demonstrated the end-to-end concept, its major responsibilities were co-located in one application and its files and generated artifacts were tied to the local filesystem. Stage 2 investigates how that prototype can be evolved into a remotely deployable, cloud-oriented architecture without discarding the validated execution core. The new system separates a thin .NET Agent from an ASP.NET Core Model Context Protocol (MCP) server. The Agent connects through MCP over HTTP, discovers server tools dynamically, and uses GPT-4o at runtime. The server exposes seven file/chart tools, owns Python validation and execution, and stores inputs and generated images in Azure Blob Storage. Blob paths are scoped by a sanitized team prefix, input data are cached locally when Python requires a filesystem path, and generated images are returned through time-limited Shared Access Signature (SAS) URLs. A multi-stage Docker image packages .NET 10 together with Python and the required visualization/data libraries, and the supplied deployment transcript records successful deployment to Azure Container Apps after iterative troubleshooting of registry authentication and regional environment constraints. The Stage 2 source contains 38 MSTest methods; the authors confirmed that all 38 passed in the final project environment. AI assistance from ChatGPT and Claude was used for design, implementation guidance, testing, and deployment debugging, while the MCP architecture and integration were a significant manual contribution. The results show a clear transition from a local monolith to a reusable client/server service, while also exposing trade-offs in authentication, sandbox strength, upload limits, SAS lifetime, and regression coverage.

Index Terms—Model Context Protocol, AI agent, cloud computing, Azure Container Apps, Azure Blob Storage, .NET 10, GPT-4o, Python, code generation, data visualization, AI-assisted development.

I. INTRODUCTION

A. Problem Statement

Large language models can generate useful data-analysis and visualization code from natural-language instructions, but generating code and executing it are different engineering problems. A dependable system has to decide which data the model is allowed to see, how generated code is checked, where

it is executed, how results are stored, and how failures are reported. Prior work on code-generating models established the feasibility of translating natural language to executable code, while work on tool-using agents demonstrated the value of connecting language models to external actions and APIs [1]–[5]. The practical challenge addressed by this university project is therefore not merely chart creation: it is the orchestration of an LLM, files, generated Python, validation, execution, and persistent artifacts in a maintainable architecture.

The first project stage solved this problem as a local prototype. A .NET console agent used GPT-4o, exposed chart/file functionality through an in-process plugin, analyzed local datasets, validated generated Python, launched Python as a child process, persisted error information, and recorded generated charts. This demonstrated the complete natural-language-to-chart loop, including large-file handling and representative visualization scenarios [16]. However, the Stage 1 architecture coupled the conversational agent to the execution/storage environment. A local file path could be passed directly to the same application, generated charts were local artifacts, and the tool implementation did not form an independently deployable service.

Stage 2 asks a different question: *How can the validated Stage 1 execution concept be transformed into a cloud-deployable service that can be called remotely by an AI agent?* The answer implemented in Folder2 is an architectural refactor centered on the Model Context Protocol. MCP provides a client/server mechanism in which servers expose named tools with schemas that clients can discover and invoke [7], [8]. Microsoft Agent Framework likewise supports agents that use LLMs and call MCP servers/tools [9], [10]. These capabilities make it possible to move file management and Python execution out of the local Agent while keeping the Agent focused on conversational orchestration.

B. Stage 2 Goals and Scope

The Stage 2 implementation has six principal goals:

- 1) separate the AI client from the execution/storage service;
- 2) expose the project functions through MCP over HTTP rather than direct in-process plugin calls;

- 3) replace local persistence with Azure Blob Storage and return remotely accessible chart results;
- 4) introduce team-scoped storage namespaces so one server can separate multiple groups of files;
- 5) package the mixed .NET/Python runtime as a Docker image and deploy it to Azure Container Apps; and
- 6) preserve the useful Stage 1 Python validation/execution logic where possible, instead of rewriting already tested behavior solely because deployment changed.

The scope is intentionally educational rather than production-complete. Stage 2 does not add a strong identity provider, does not claim that the Python process is a formally secure sandbox, and does not contain a complete live-cloud end-to-end test suite. These limitations are treated explicitly in the Discussion because the course instructions require correctness/quality issues in AI-assisted output to be documented rather than hidden.

C. Contributions

The main contribution is a concrete evolution path from a local AI application to a cloud-oriented MCP service. The resulting system combines a thin Agent, dynamic MCP tool discovery, an ASP.NET Core tool server, Azure Blob Storage, a local server cache for Python-readable files, SAS-based chart delivery, Docker packaging, and Azure Container Apps deployment. A second contribution is methodological: the work records how ChatGPT and Claude were used as development assistants while preserving a distinction between surviving verbatim prompts and reconstructed representative prompts when historical chat data was not retained.

II. METHODS

A. Project Audit and Stage-to-Stage Comparison Method

Before writing the Stage 2 design, both project folders were inventoried and compared. The supplied archive contains 643 files: 375 in Folder1 and 268 in Folder2. Many are generated Visual Studio, `bin`, or `obj` artifacts, so architectural comparison focused on authored source/configuration/documentation while still checking generated artifacts for stale or contradictory evidence. Excluding `bin`, `obj`, and `.vs`, the Stage 1 tree contains 24 authored C# files (approximately 3,403 source lines), while Stage 2 contains 19 authored C# files (approximately 2,791 source lines). The reduction is meaningful: Stage 2 removes several Stage 1 subsystems while moving core execution functionality behind a server boundary.

The comparison was class- and responsibility-oriented rather than based only on filenames. Components were categorized as (1) retained/moved, (2) replaced, (3) removed, or (4) newly introduced. This avoided the misleading conclusion that every deleted file represented lost functionality; for example, Stage 1's local chart manifest is partly replaced by listing output blobs, whereas the rich `DatasetAnalyzer` has no equivalent implementation in Stage 2.

B. Stage 1 Baseline

The Stage 1 application establishes the baseline from which Stage 2 evolved. Its main `Program.cs` configured the agent, Python path, storage, validator, executor, error-memory store, chart manifest, and plugin. Because these dependencies lived in the same process, the agent could pass local file paths directly to helper components.

The `DatasetAnalyzer` was important for large inputs: it sampled CSV data, inferred column characteristics, produced compact descriptions, and prevented the model from needing the entire large file in its prompt. `PythonCodeValidator` applied disallowed-import/function checks and chart-structure expectations, then invoked Python AST parsing for syntax validation. `PythonExecutor` wrote/runs scripts as child processes, captured output and errors, and applied time/resource controls. `ErrorMappingStore` persisted categorized error records to JSON and could summarize recent errors back into model context. `ChartManifest` stored chart history. This Stage 1 design provided a rich local experience but kept storage and execution tightly coupled to the console application.

C. Stage 2 Runtime Architecture

Figure 1 shows the implemented Stage 2 runtime path. The local Agent remains responsible for the conversational interaction and model call. The server is the authoritative execution environment. This separation follows the MCP model in which tools are functions exposed by a server and discoverable by clients [7].

The Agent's `McpHttpTransport.RunAsync()` reads the API key, runtime model, server endpoint, and team value from configuration. It creates an MCP HTTP client and adds `Team-Name` as an additional request header. After the MCP handshake it calls `ListToolsAsync()` and converts discovered MCP tools into AI tools. This is materially different from Stage 1: the Agent does not compile against and instantiate the server's chart implementation directly.

The server registers `ChartPlugin`, `IStorageStore`, `PythonCodeValidator`, `PythonCodeExecutor`, HTTP context access, and the Python configuration through ASP.NET Core dependency injection. It registers the MCP server with HTTP transport, binds `ChartPlugin` as the server tool type, and maps the MCP endpoint. MCP's current transport specification defines `stdio` and `Streamable HTTP` as standard transports and uses JSON-RPC messages for client/server communication [8]. Folder2 contains an experimental `McpStdioTransport.cs`, but the operational design is HTTP; the current ASP.NET Core server is an independent remotely hosted process rather than a child process started over standard input/output.

D. MCP Tool Surface

The Stage 2 server exposes seven tools through `ChartPlugin`:

- 1) list uploaded input files;
- 2) delete uploaded input files;
- 3) preview an uploaded file;

TABLE I: Architectural evolution from Stage 1 to Stage 2

Concern	Stage 1 (Folder1)	Stage 2 (Folder2)	Change type
Application topology	One local .NET console application; Agent and chart plugin operate in-process	Separate Agent and ASP.NET Core MCP server applications	Major architectural refactor
Tool invocation	Direct ChartPlugin calls exposed to the local agent	MCP over HTTP; server tools discovered at runtime	New remote protocol boundary
Storage	LocalFileStore and local paths	AzureBlobStorageStore; private input/output containers	Local-to-cloud replacement
User/team separation	No team namespace	Sanitized Team-Name used as blob prefix	New logical isolation
File transfer	Local path directly accessible to application	Local Agent reads file, enforces 50 MB limit, Base64 encodes, calls server upload	New client/server adapter
Chart result	Local PNG path plus local manifest	Output blob plus SAS URL; blob listing for history	Remote artifact delivery
Dataset profiling	DatasetAnalyzer with sampling/schema/type inference	No equivalent rich analyzer; preview is simpler	Capability removed/reduced
Error memory	ErrorMappingStore persists categorized errors	No equivalent persistent error-learning store	Removed
Execution core	Validator and Python child-process executor	Same architectural core moved into server namespace	Largely reused/moved
Python setup	Python path resolution	Readiness/setup component checks Python/packages	Expanded operational support
Deployment	Local execution	Multi-stage Docker image, ACR, Azure Container Apps	New cloud deployment
Tests	38 test methods plus five dynamic golden scenarios	38 test methods; no active golden-scenario source suite	Coverage emphasis changed

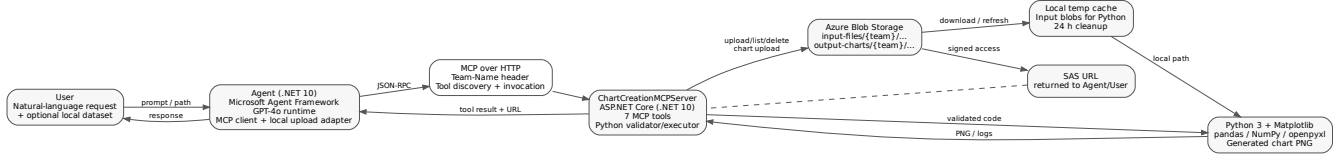


Fig. 1: Stage 2 runtime architecture. The Agent communicates with a remote MCP server over HTTP; storage and Python execution are server responsibilities.

- 4) upload file content;
- 5) resolve a blob-backed input to a local cached path;
- 6) validate and generate a chart from Python code; and
- 7) list generated charts stored for the team.

This tool set gives the model control over a narrow application-specific interface rather than arbitrary shell access. The `GenerateChart` tool checks Python readiness, validates code, replaces an `OUTPUT_PATH` placeholder with a server-controlled file path, executes the script, uploads the resulting PNG to Blob Storage, returns a SAS URL, and removes temporary script/image files on a best-effort basis.

MCP tool discovery is especially useful for decoupling: tool names, descriptions, and parameter schemas come from the server rather than being duplicated in the Agent. Microsoft documentation describes Agent Framework as supporting agents that call tools and MCP servers, and its MCP integration can retrieve available server tools for agent use [9], [10].

E. Local Upload Adapter

A direct remote migration creates a non-obvious problem: a path such as `C:\Users\...\sales.csv` exists on the user’s machine, not inside Azure. The Stage 1 method of

handing a local path to the chart tool cannot therefore survive the client/server split.

Stage 2 solves this in the Agent’s `Helpers.cs`. The Agent deliberately excludes the server’s raw upload tool from the list exposed to the model and adds a local function with the user-friendly signature `UploadFileAsync(filePath)`. The wrapper verifies that the local file exists, enforces a 50 MB maximum, reads its bytes, encodes the content, and calls the server upload tool. This preserves a simple model-facing workflow while explicitly crossing the machine boundary. The 50 MB cap is also a Stage 2 limitation: it is lower than the very-large-file scenario emphasized in Stage 1.

F. Azure Blob Storage and Team Namespaces

Stage 2 implements `IStorageStore` with `AzureBlobStorageStore`. The constructor creates/uses two private Blob containers, conceptually `input-files` and `output-charts`. Azure’s .NET Blob client supports the upload, download, list, and SAS-related operations required by this implementation [12]. Blob Storage was selected because the server needs persistence independent of any single container replica or local disk.

Every storage method sanitizes the team identifier and constructs blob names under a team prefix. In simplified form, the paths are:

```
input-files/{team}/sales.csv
output-charts/{team}/sales_trend_....png
```

The `ChartPlugin.CurrentTeam()` method reads `Team-Name` from the request and uses `default` when the header is absent. This is useful logical namespace separation, but it is not authentication: the server does not independently verify that the caller is entitled to a supplied team value. This distinction is revisited in Section V.

The storage implementation also bridges object storage and Python’s filesystem expectation. When Python needs a source file, the server downloads the blob into a local temporary cache. The cached copy is compared with the blob’s `LastModified` information and refreshed when needed. A hosted background service removes stale cache files every 24 hours; cloud blobs are not deleted by this cleanup. If a cleaned file is later requested, it is downloaded again.

Generated charts are uploaded to the output container and the server returns a SAS URL. The current code/README uses a long-lived sharing period (approximately one year). SAS makes remote retrieval convenient, but the resulting URL is a bearer credential for its validity period; Microsoft recommends careful credential handling and supports identity-based/user-delegation approaches for stronger designs [12].

G. Python Validation and Execution

A deliberate Stage 2 choice was to preserve the Stage 1 validation/execution core rather than rewrite it simply because a network boundary was added. `PythonCodeValidator.cs`, `SandboxConfig.cs`, and the executor are structurally very similar between stages, with namespace/integration adjustments for the server. This is an example of architecture evolution rather than feature replacement.

The validator rejects a set of dangerous imports/functions, checks chart-related structural expectations, and validates Python syntax. It should not be described as a complete security proof. In particular, the safety layer is largely pattern-based, while Python AST parsing is used principally to verify syntax. Obfuscated imports or dynamic behavior can defeat simple blocked-string approaches. Likewise, the child-process executor provides timeout/output/resource handling, but the source does not establish strong platform-independent network isolation or Linux cgroup-level memory isolation by itself.

H. Python Environment Readiness and Health

Stage 2 replaces the simpler Python path resolver with `PythonEnvironmentSetup`. At server startup it checks the Python executable/version and the required packages. Depending on configuration it can attempt installation, but the Docker deployment disables startup auto-install because the image already contains dependencies. The server exposes a `/health` endpoint containing overall and Python readiness information, enabling diagnostics without interactive shell access.

I. Containerization and Azure Deployment

The Dockerfile uses two stages. The first uses the .NET 10 SDK to restore and publish the ASP.NET Core project. The second uses the .NET 10 ASP.NET runtime, installs Python 3 and `matplotlib`, `pandas`, `numpy`, and `openpyxl`, copies only the published .NET output, and exposes port 8080. Docker documents multi-stage builds as a mechanism for separating build and runtime stages and copying only required artifacts into the final image [13]. .NET 10 is an LTS release [14].

The deployment target confirmed for Stage 2 is Azure Container Apps. Container Apps is a managed/serverless platform for containerized applications with external/internal ingress, registry integration, revisions, logging, and scaling capabilities [11]. The supplied terminal transcript records the practical deployment sequence: local Docker build/run, Azure Container Registry login and push, failed/abandoned App Service attempts, Container Apps environment creation, and final container app creation with external ingress to port 8080.

Figure 2 summarizes the evolution and deployment boundary.

J. AI-Assisted Development Workflow

AI assistance was used throughout both stages. Stage 1 has a preserved ChatGPT planning prompt and raw response in the submitted appendix [17]. Stage 2 used both ChatGPT and Claude, but the complete original conversations and exact model versions were not retained. The supplied deployment notes preserve a general instruction context requiring the AI to act as a senior software engineer, avoid guessing, reason only from supplied code/logs, request missing files, explain reasoning, and prefer maintainable solutions [18].

The practical workflow was iterative:

- 1) give the AI the current source/problem rather than a generic description;
- 2) request a narrow architectural or coding change;
- 3) integrate or modify the suggestion manually;
- 4) compile/run tests or execute the relevant Azure/Docker command;
- 5) return the exact error/log to the AI for diagnosis;
- 6) apply the smallest corrective change and repeat; and
- 7) review documentation claims against the code before submission.

The most significant manually developed Stage 2 area was the MCP architecture: deciding the client/server responsibility boundary, deciding which Stage 1 components to move/reuse/remove, and integrating file transfer, headers, dependency injection, storage, and deployment. AI accelerated implementation and troubleshooting but did not replace these design decisions.

K. Evaluation Method

Evaluation used three evidence types: source-level tests, deployment/runtime logs, and architectural inspection. The Stage 2 test project contains 38 `[TestMethod]` methods. The authors confirmed that all 38 passed in the final Stage 2 environment. The supplied deployment transcripts do not

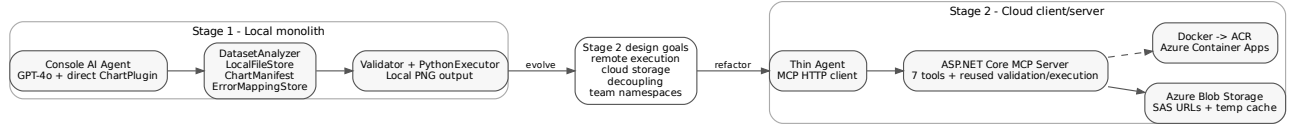


Fig. 2: Evolution from the Stage 1 local monolith to the Stage 2 MCP/cloud architecture.

TABLE II: Stage 2 test-method distribution

Test class	Methods
PythonCodeValidatorTests	7
PathSafetyTests	11
PythonCodeExecutorTests	5
AzureBlobStorageStoreTests	1
ChartPluginTests	14
Total	38

contain the dotnet test output, so this paper distinguishes author-confirmed pass status from independently visible terminal evidence.

The test distribution is shown in Table II. It demonstrates a shift away from Stage 1’s local file/error/golden-scenario components toward path/team safety and MCP tool behavior.

III. RESULTS

A. Implemented System

The principal result is a functioning separation of conversational orchestration from chart execution. The Agent can establish an HTTP MCP session, discover server capabilities, provide them to the runtime GPT-4o agent, and maintain an interactive Agent Framework session. The server owns all seven application tools and can operate without sharing implementation assemblies with the client.

The Stage 2 data path is also cloud-oriented. An input chosen on the client is uploaded as content rather than as an unusable remote filesystem path. The server persists it to Blob Storage under a team prefix, resolves a local cached path for Python, validates/runs generated visualization code, uploads the PNG to output storage, and returns a remote URL. Thus the chart artifact survives the temporary execution path and can be accessed independently of the server’s local filesystem.

B. Deployment Evidence

The two supplied deployment logs provide concrete evidence beyond source inspection. First, the Docker build completed successfully and the image could be started locally. The runtime reported Python 3.12.3 and the expected packages (Matplotlib, pandas, NumPy, and openpyxl) as ready; the cleanup service started and ASP.NET Core listened on port 8080. These observations directly validate the mixed .NET/Python image design.

Second, the initial container-registry push failed because authentication was required. After an explicit Azure Container Registry login, the push succeeded and a content digest was reported. This is useful evidence of an AI-assisted debugging

cycle because the final state was reached through inspection and correction rather than a single idealized deployment command.

Third, the transcript shows that an early App Service route was attempted but was not the final architecture. Container Apps environment creation also encountered subscription/region constraints. A Germany West Central environment attempt ultimately produced a maximum-environments error; the supplied transcript then records successful environment creation in France Central. Finally, Azure reported successful creation of app-chartserver from the chartserver:v1 image with external ingress, target port 8080, 0.5 vCPU, and 1.0 GiB memory, and returned an azurecontainerapps.io endpoint [18]. The active source configuration also points to an azurecontainerapps.io server endpoint, confirming Container Apps as the intended hosting service even if the exact endpoint/region changed after the recorded transcript.

For security, this paper intentionally omits the registry password and other secrets that appear in the raw terminal history. Such credentials should be rotated/redacted before the raw logs are submitted or published.

C. Test Results

All 38 Stage 2 test methods were reported by the authors as passing. The distribution in Table II shows that the largest groups target ChartPlugin (14) and path safety (11), followed by validation and executor behavior. This is consistent with the refactor: the main new risk is correct delegation/scoping across server/tool/storage boundaries, while the validation/execution core retains regression tests.

The Stage 2 test suite does not, however, reproduce all Stage 1 evaluation assets. Folder1 contains 38 test methods plus a dynamic golden-scenario test driven by five visualization scenarios; Folder2 no longer contains those golden-scenario source files. Some compiled Folder2 artifacts still contain older scenario files, but they are stale build output and are not treated as active Stage 2 source. Therefore the Stage 2 result is strong at unit-level refactoring checks but weaker in explicit source-controlled visualization scenario regression.

D. Functional and Architectural Differences

Table I demonstrates that Stage 2 is not simply “Stage 1 plus Azure.” Several capabilities were intentionally removed or replaced. DatasetAnalyzer is absent, so Stage 2 does not provide the same rich automatic dataset profiling. ErrorMappingStore is also absent, so runtime failures can still be returned to the current model interaction but are not accumulated in the same persistent learning store. ChartManifest is replaced by cloud chart listing rather

than a local JSON manifest. Conversely, Stage 2 adds protocol decoupling, cloud persistence, team namespaces, remote result delivery, Python health visibility, containerization, and managed cloud deployment.

The result is therefore a trade: Stage 2 improves deployability, separation of concerns, persistence, and remote accessibility while reducing some of Stage 1’s local data-intelligence and error-memory features. This distinction matters academically because an architecture migration should be evaluated by both gains and regressions.

IV. AI USAGE AND DISCUSSION

A. Stage 1 AI Evidence

The strongest surviving AI evidence is the Stage 1 appendix. It explicitly records use of OpenAI’s ChatGPT and includes the original project-planning prompt. That prompt described a Microsoft Agent Framework project in .NET/C#, an AI agent generating Python visualization code, a custom code-interpreter tool, large-data concerns, and safe/reliable code execution, and then asked ChatGPT to propose how the project should be developed [17]. The raw response proposed an architecture with an agent, dataset profiler, validator, Python sandbox runner, artifact management, phased development, and evaluation ideas.

The final Stage 1 code was not a verbatim copy of that response. The proposal had to be translated into concrete classes, framework APIs, storage behavior, validator rules, tests, and error/chart persistence. This is a representative example of effective AI-assisted engineering: the AI supplied decomposition and implementation ideas, while the developers decided how those ideas mapped to a real codebase.

B. Stage 2 AI Evidence and Attribution Limits

The authors confirmed that ChatGPT and Claude were the main development assistants in Stage 2. However, the full task-specific chat transcripts, exact model versions, and Git history were not retained. It would therefore be inaccurate to invent a tool/version for each method or to claim reconstructed text was a verbatim prompt. The separate `prompt.md` log supplied with this report marks each entry either as retained verbatim evidence or as a reconstructed representative prompt.

The surviving Stage 2 instruction context is still informative. It required the AI to avoid hallucination, work only from supplied code/information, ask for exact files when context was insufficient, explain reasoning, and prefer the simplest maintainable solution. The deployment terminal history shows how that instruction was operationalized: real Azure errors were fed into an iterative troubleshooting process, and commands were corrected until the registry push and Container Apps deployment succeeded.

C. Correctness/Quality Issues Found in AI Output

The requirement to discuss AI-output quality is important because code assistants can produce technically plausible but overconfident explanations. Several claims need qualification when compared with the source:

1) Validation is not a complete sandbox. The validator blocks many dangerous patterns and checks syntax, but string/pattern rules can be bypassed by dynamic or obfuscated Python. A production design would require stronger isolation and more semantic/AST-level policy enforcement.

2) Network isolation is not proven. The executor sets environment values including `NO_PROXY=*`; this does not mean all outbound networking is blocked. Documentation should therefore avoid calling the process fully network-isolated.

3) Team separation is not authentication. The `Team-Name` header provides a useful namespace but can be supplied by the caller. MCP’s HTTP transport guidance explicitly recommends proper authentication for remote connections [8]. A production deployment should authenticate users/teams and derive authorization server-side.

4) Resource controls are platform-dependent. The child process has timeout/resource handling, but strong Linux container memory isolation should be enforced through container/platform limits rather than assumed from Windows-oriented working-set behavior.

5) SAS lifetime trades convenience for exposure. A one-year bearer URL is easy to revisit/share but remains valid if leaked. Shorter SAS lifetimes, user delegation SAS, authenticated download endpoints, or per-request authorization would reduce exposure.

6) Unit tests do not prove live cloud integration. The source unit tests are valuable, but they do not constitute a complete real-Azure end-to-end suite. The deployment logs provide deployment/runtime evidence, not systematic functional load/security evaluation.

V. DISCUSSION

A. What Worked Well

The strongest Stage 2 improvement is separation of concerns. The Agent no longer needs to know how Blob Storage works or how Python is launched. Conversely, the server does not need to own the user conversation or model API interaction. This reduces coupling and creates a reusable service boundary: another MCP-capable client could theoretically call the same server tools without embedding the server implementation.

Dynamic tool discovery also reduces duplication. The Agent receives tool metadata from the server rather than maintaining a second manually synchronized tool schema. This follows MCP’s intended model of server-advertised tools [7]. The upload wrapper demonstrates an effective exception: when a tool needs client-local access, the Agent can adapt a remote primitive into a model-friendly local function without breaking the broader protocol architecture.

Cloud storage solves an important persistence problem. Container-local disks are ephemeral and unsuitable as the only source of truth for user files or generated charts. Blob Storage decouples artifacts from container lifetime and makes chart history available across sessions. The local cache retains the practical performance/compatibility benefit of a filesystem path for pandas/Python without making local disk authoritative.

TABLE III: Representative AI-assisted contributions and manual review

Contribution	AI assistance	Surviving evidence	Manual engineering/review
Stage 1 architecture	Initial system decomposition and development plan	Verbatim ChatGPT prompt + raw response in Stage 1 appendix	Converted proposal to concrete C# architecture, integrations, tests, and project constraints
MCP refactor	Design/code suggestions for client/server split and MCP calls	Final MCP client/server source; exact task chat not retained	Major manual contribution: responsibility boundary, component reuse/removal, integration and debugging
Cloud storage	Suggestions for Blob Storage, namespace/scoping, cache and URLs	Azure Blob storage implementation and tests	Implemented/sanitized team prefixes, cache semantics, private containers, SAS behavior
Remote upload	Suggestions for moving local file content to remote server	Agent upload wrapper and server upload tool	Selected client-side adapter and 50 MB guard; maintained model-facing simplicity
Docker/deployment	Dockerfile/CLI guidance and error interpretation	Dockerfile plus deployment logs	Rebuilt image, authenticated registry, changed service/region approach, verified runtime
Testing	Edge-case/test ideas and refactor assistance	38 Stage 2 MSTest methods	Aligned tests with real interfaces/mocks and executed final suite

The Docker image also makes the mixed technology stack reproducible. Stage 1 required a suitable Python environment on the machine. Stage 2’s runtime image includes the .NET server and Python packages in one deployable unit. The deployment transcript validates that the container boots with the expected Python environment and HTTP listener.

Finally, the troubleshooting history is valuable evidence of realistic cloud engineering. The final architecture was not obtained by copying one perfect command. Registry authentication, an abandoned App Service path, policy/region constraints, and environment limits had to be diagnosed. AI was useful as an interactive debugging assistant precisely because it could interpret the latest concrete error, but every proposed fix still required manual execution and verification.

B. What Did Not Carry Forward

The cloud refactor also introduced regressions. Stage 1’s `DatasetAnalyzer` performed richer data inspection and sampling; Stage 2 relies more on simple preview instructions and model-generated pandas code. This may reduce robustness for unfamiliar schemas or very large data, particularly because the Agent’s upload wrapper caps ordinary uploads at 50 MB.

The persistent `ErrorMappingStore` was removed. As a result, Stage 2 can retry based on immediate tool errors and conversational context, but it no longer has the same persisted categorized error memory. This simplifies the server but sacrifices a specific self-improvement mechanism from Stage 1.

Similarly, the source-controlled golden-scenario suite is absent. This makes it harder to demonstrate that representative chart workflows still behave identically after the architecture migration. Reintroducing those scenarios as MCP end-to-end tests would be a high-value future improvement.

C. Security and Multi-Tenancy Limitations

The current `Team-Name` mechanism should be described as namespace partitioning, not tenant security. A malicious

caller who can choose arbitrary headers may be able to request another logical prefix. The next design should authenticate the caller (for example with Microsoft Entra ID or another identity provider), map the authenticated identity to a server-controlled team/tenant identifier, and authorize each storage action.

Secrets also require improvement. The raw project history contains examples of plaintext credentials in configuration/logs. Even if such values were temporary, they should be removed from submission artifacts and rotated. Azure Container Apps can manage application secrets, while managed identity can reduce dependence on static registry/storage credentials [11]. The storage layer should ideally use identity-based credentials rather than a long-lived connection string where feasible.

Generated-code execution remains the most sensitive boundary. A practical production system should combine static checks with a hardened runtime: non-root containers, read-only filesystem where possible, strict egress controls, CPU/memory limits enforced by the platform, minimal Python dependencies, syscall/security profiles, and potentially a separate short-lived execution worker rather than executing inside the long-running MCP API process.

D. Scalability and Operational Considerations

Azure Container Apps is suitable for a small API/service because it provides managed ingress and automatic scaling options [11]. Nevertheless, the current Python execution architecture is synchronous and local to the server replica. Heavy chart requests may consume CPU/memory for the duration of execution. If traffic grows, a queue/worker design could decouple MCP request handling from long-running jobs. Generated-code execution could run in disposable worker containers with explicit resource limits, with the API returning job status/result URLs.

The local cache also needs multi-replica awareness. Because Blob Storage is authoritative this is functionally safe, but each replica may download its own copy. Large datasets could

therefore increase cold-start/network/storage costs. A bounded cache with size limits, access-based eviction, metrics, and workload-specific policies would be more predictable than only age-based cleanup.

E. Lessons Learned

Several lessons emerge from the Stage 1 to Stage 2 evolution. First, cloud migration is primarily a boundary problem, not a hosting problem. The difficult change was making local assumptions explicit: file paths, artifact lifetime, tool calls, and user identity. Second, not every Stage 1 component should be rewritten. Reusing the validator/executor reduced risk, while storage and orchestration appropriately changed. Third, AI assistants are most effective when given exact source/log context and asked for narrow changes. The deployment transcript shows that generic cloud advice is less useful than reasoning from the current CLI error. Fourth, AI-generated documentation must be validated against code; security terms such as “sandbox”, “isolation”, and “multi-tenant” can easily become stronger than the implementation actually proves.

F. Future Work

Future work should prioritize: (1) authenticated identity and authorization instead of trusting `Team-Name`; (2) managed identity for Azure services and shorter/safer result authorization; (3) hardened short-lived Python execution workers with platform-enforced resource/network policies; (4) restoration of rich dataset profiling and large-file strategies; (5) reintroduction of persistent error-learning if pedagogically useful; (6) MCP-level end-to-end and live Azure integration tests, including the five representative golden scenarios; (7) observability such as structured logs, metrics, tracing, execution duration/error rates, and storage/cache metrics; and (8) CI/CD that automatically builds, tests, scans, pushes, and deploys immutable image versions.

VI. CONCLUSION

Stage 2 successfully evolves the Python Code Tool Interpreter for Chart Creation from a local, tightly coupled AI application into a cloud-oriented client/server system. The most important change is architectural: a thin .NET Agent connects over MCP/HTTP to an independent ASP.NET Core server that exposes seven application tools, owns storage and Python execution, and returns cloud-hosted chart results. Azure Blob Storage replaces the local filesystem as durable persistence, team prefixes provide logical separation, a local cache bridges blob objects to Python file paths, and a multi-stage Docker image packages .NET 10 with Python and the required visualization libraries. The supplied terminal evidence records successful container startup, registry push, and Azure Container Apps deployment after several realistic troubleshooting iterations.

The project also demonstrates that architectural improvement is not automatically feature expansion. Stage 2 removes Stage 1’s rich dataset analyzer, persistent error-learning component, chart manifest, and active golden-scenario source suite. The resulting system is more deployable and decoupled but has

new constraints such as the 50 MB Agent upload limit and remote-security requirements. The 38 Stage 2 test methods were confirmed by the authors as passing, but stronger cloud end-to-end, security, and performance evaluation remains future work.

ChatGPT and Claude materially assisted planning, coding, tests, and deployment debugging. However, the final engineering outcome depended on manual MCP architecture decisions, integration, verification, and correction of overconfident AI claims. This combination of AI acceleration with explicit human validation is the central methodological lesson of the project.

REFERENCES

- [1] M. Chen et al., “Evaluating large language models trained on code,” arXiv:2107.03374, 2021.
- [2] P. Yin and G. Neubig, “A syntactic neural model for general-purpose code generation,” in *Proc. ACL*, 2017, pp. 440–450.
- [3] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, “Mapping language to code in programmatic context,” in *Proc. EMNLP*, 2018, pp. 1643–1653.
- [4] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023/2024.
- [5] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” *NeurIPS*, vol. 36, 2023.
- [6] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [7] Model Context Protocol, “Tools - Protocol Revision 2025-06-18,” 2025. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-06-18/server/tools>. Accessed: Aug. 18, 2026.
- [8] Model Context Protocol, “Transports - Protocol Revision 2025-06-18,” 2025. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-06-18/basic/transports>. Accessed: Aug. 18, 2026.
- [9] Microsoft, “Microsoft Agent Framework Overview,” Microsoft Learn, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/agent-framework/overview/>. Accessed: Aug. 18, 2026.
- [10] Microsoft, “Get started with .NET AI and the Model Context Protocol,” Microsoft Learn, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/ai/get-started-mcp>. Accessed: Aug. 18, 2026.
- [11] Microsoft, “Azure Container Apps overview,” Microsoft Learn, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/container-apps/overview>. Accessed: Aug. 18, 2026.
- [12] Microsoft, “Get started with Azure Blob Storage and .NET,” Microsoft Learn, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/azure/storage/blobs/storage-blob-dotnet-get-started>. Accessed: Aug. 18, 2026.
- [13] Docker, “Multi-stage builds,” Docker Documentation, 2026. [Online]. Available: <https://docs.docker.com/build/building/multi-stage/>. Accessed: Aug. 18, 2026.
- [14] Microsoft, “What’s new in .NET 10,” Microsoft Learn, 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview>. Accessed: Aug. 18, 2026.
- [15] OpenAI, “GPT-4o Model,” OpenAI API Documentation. [Online]. Available: <https://developers.openai.com/api/docs/models/gpt-4o>. Accessed: Aug. 18, 2026.
- [16] M. A. Rahman and M. S. Rana, “Python Code Tool Interpreter for Chart Creation,” Software Engineering project report, Frankfurt University of Applied Sciences, 2026. Supplied in Folder1.
- [17] M. A. Rahman and M. S. Rana, “Project Appendix - AI planning prompt and response,” Software Engineering project, Frankfurt University of Applied Sciences, 2026. Supplied in Folder1 documentation.
- [18] M. A. Rahman and M. S. Rana, “Stage 2 Azure/Docker deployment and troubleshooting transcripts,” files `azure dep.txt` and `azure dep mcp.txt`, Aug. 2026. Private project evidence; credentials redacted in this report.